

WIA2005: Algorithm Design and Analysis

Lecture 6: Heap Sort

Asmiza A. Sani

Semester 2, Session 2025/26

Heapsort Algorithm

- Like merge sort, but unlike insertion sort, heapsort's running time is $O(n \log n)$.
- Like insertion sort, but unlike merge sort, heapsort sorts in place: only a constant number of array elements are stored outside the input array at any time.
- *Thus, heapsort combines the better attributes of the insertion and merge sort.*
- Heapsort also introduces another algorithm design technique: using a data structure, which we call a “heap,” to manage information.

Heap

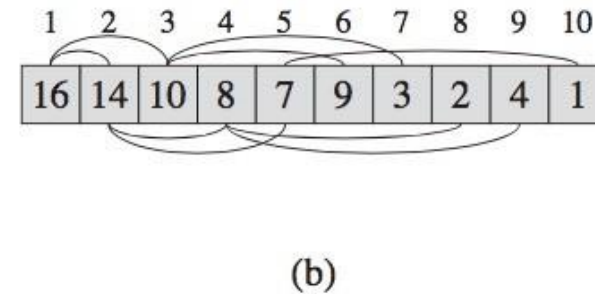
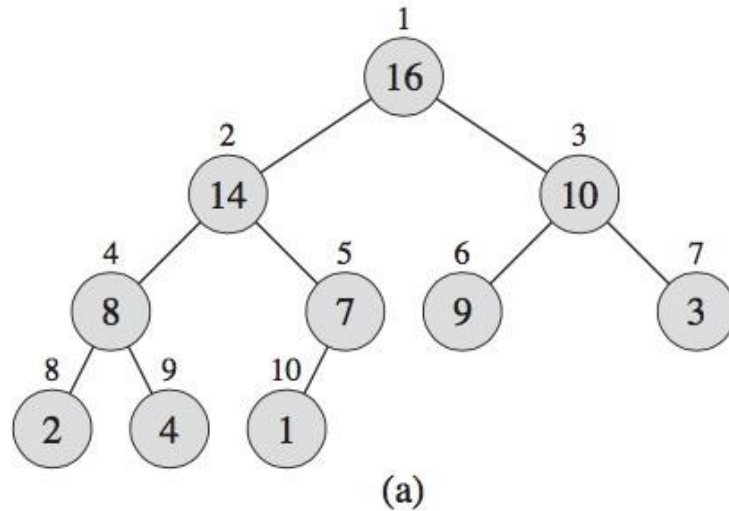
- The *(binary) heap* data structure is an array object that we can view as a nearly complete binary tree.
- Each node of the tree corresponds to an element of the array.
- The tree is filled on all levels except possibly the lowest, which is filled from the left up to a point.
- An array *A* that represents a heap is an object with two attributes:
 - *A.length*, which (as usual) gives the number of elements in the array.
 - *A.heap-size*, which represents how many elements in the heap are stored within array *A*.

Heaps property

- There are two kinds of binary heaps:
 - max-heaps
 - min-heaps
- Max-heap property: $A[\text{PARENT}(i)] \geq A[i]$
- Min-heap property: $A[\text{PARENT}(i)] \leq A[i]$

Max-heap

- A max-heap viewed as (a) a binary tree and (b) an array. The number within the circle at each node in the tree is the value stored at that node. The number above a node is the corresponding index in the array. Above and below the array are lines showing parent-child relationships; parents are always to the left of their children. The tree has height three; the node at index 4 (with value 8) has height one.



Computing indices

- The root of the tree is $A[1]$, and given the index i of a node, we can easily compute the indices of its parent, left child, and right child:

PARENT(i)

1 return $\lfloor i/2 \rfloor$

LEFT(i)

1 return $2i$

RIGHT(i)

1 return $2i + 1$

- The LEFT procedure can compute $2i$ in one instruction by shifting the binary representation of i left by one bit.
- Similarly, the RIGHT procedure can quickly compute $2i+1$ by shifting the binary representation of i left by one bit position and then adding a 1 as the low-order bit.
- The PARENT procedure can compute $\lfloor i/2 \rfloor$ by shifting i right one bit position.

Basic operation on heaps

- MAX-HEAPIFY
- BUILD-MAX-HEAP
- HEAPSORT
- MAX-HEAP-INSERT, HEAP-EXTRACT-MAX, HEAP-INCREASE-KEY, and
HEAP-MAXIMUM – for priority queue

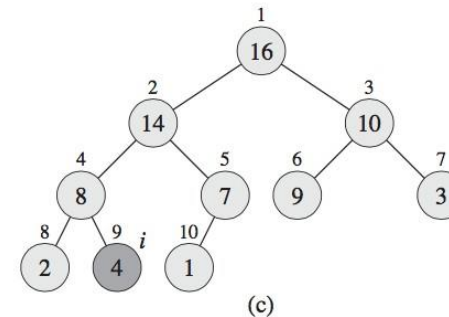
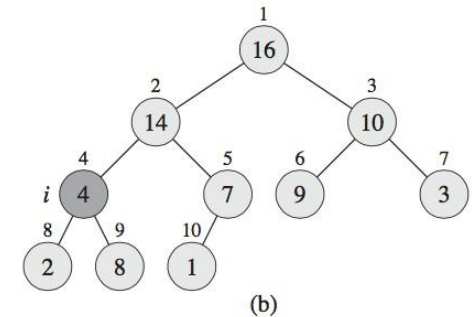
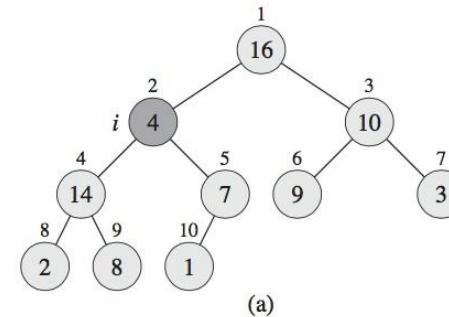
MAX-HEAPIFY: Maintaining the heap property

- To maintain the max-heap property, we call the procedure MAX-HEAPIFY.

```
MAX-HEAPIFY( $A, i$ )
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```


Visualizing MAX-HEAPIFY

- The action of $\text{MAX-HEAPIFY}(A, 2)$, where $A.\text{heap-size} = 10$. (a) The initial configuration, with $A[2]$ at node $i = 2$ violating the max-heap property since it is not larger than both children.
- Exchanging $A[2]$ with $A[4]$ restores the max-heap property for node 2 in (b) but destroys the max-heap property for node 4.
- The recursive call $\text{MAX-HEAPIFY}(A, 4)$ now has $i = 4$. After swapping $A[4]$ with $A[9]$, as shown in (c), node 4 is fixed up, and the recursive call $\text{MAX-HEAPIFY}(A, 9)$ yields no further change to the data structure.



Running time for MAX-HEAPIFY

- The running time of MAX-HEAPIFY on a subtree of size n rooted at a given node i is the $\Theta(1)$ time to fix up the relationships among the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$, plus the time to run MAX-HEAPIFY on a subtree rooted at one of the children of node i (assuming that the recursive call occurs).
- The running time of MAX-HEAPIFY by the recurrence:

$$T(n) \leq T(2n/3) + \Theta(1)$$

Using Masters Theorem

$$T(n) = \Theta(\log n)$$

- Alternatively, we can characterise the running time of MAX-HEAPIFY on a node of height h as $O(h)$.

BUILD-MAX-HEAP: Building a heap

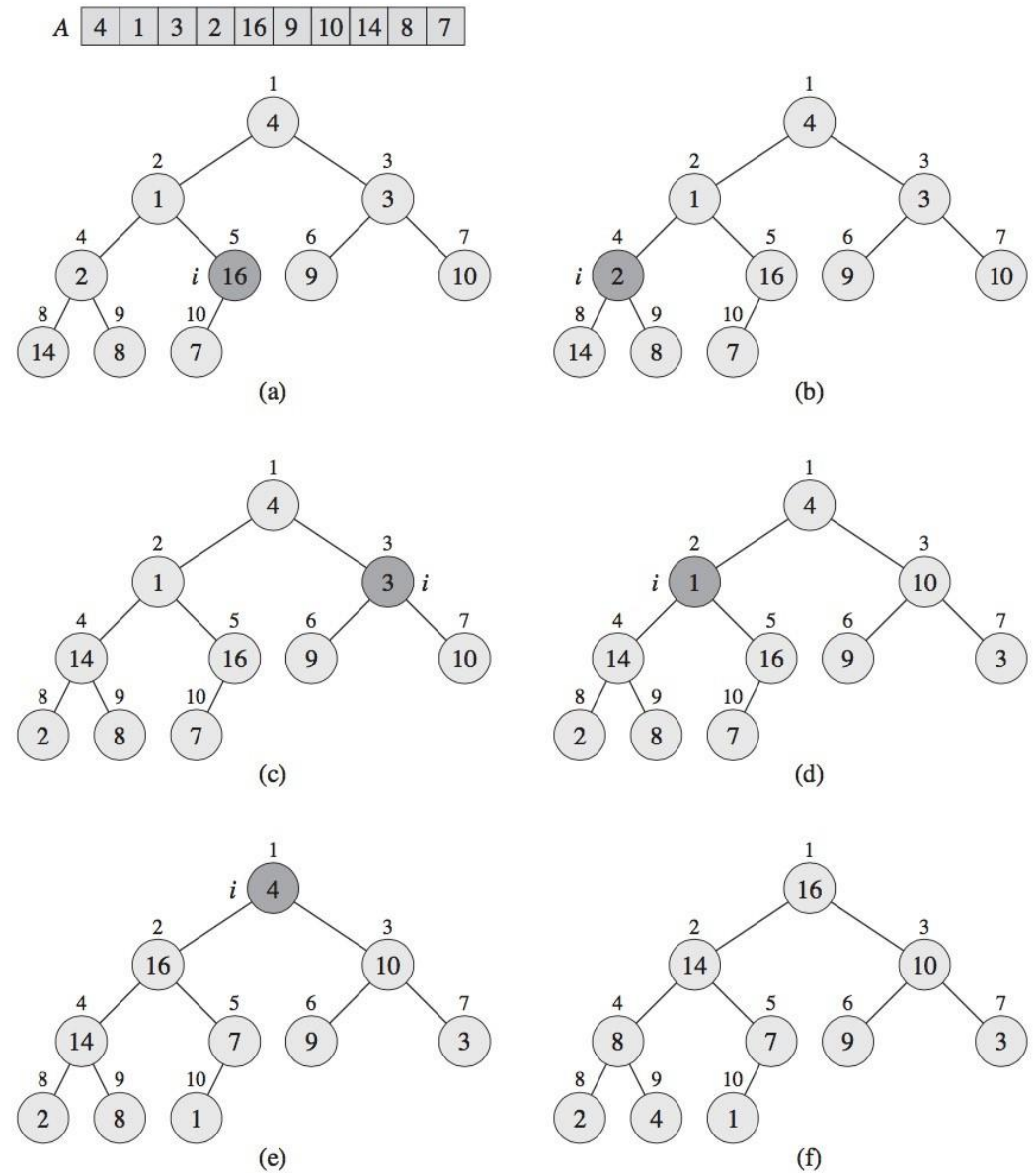
- We can use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array $A[1, \dots, n]$, where $n = A.length$, into a max-heap.

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$   
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

Visualizing BUILD-MAX-HEAP

- The operation of BUILD-MAX-HEAP, showing the data structure before the call to MAX-HEAPIFY in line 3 of BUILD-MAX-HEAP.
- (a) A 10-element input array A and the binary tree it represents. The figure shows that the loop index i refers to node 5 before the call MAX-HEAPIFY(A, i).
- (b) The data structure that results. The loop index i for the next iteration refers to node 4.
- (c)–(e) Subsequent iterations of the for loop in BUILD-MAX-HEAP. Observe that whenever MAX-HEAPIFY is called on a node, the two subtrees of that node are both max-heaps.
- (f) The max-heap after BUILD-MAX-HEAP finishes.



Running time for BUILD-MAX-HEAP

- BUILD-MAX-HEAP procedure makes $O(n)$ calls to MAX-HEAPIFY and each of the MAX-HEAPIFY calls takes $O(\log n)$.
- Therefore, the BUILD-MAX-HEAP procedure takes time $O(n \log n)$.

HEAPSORT: For sorting elements

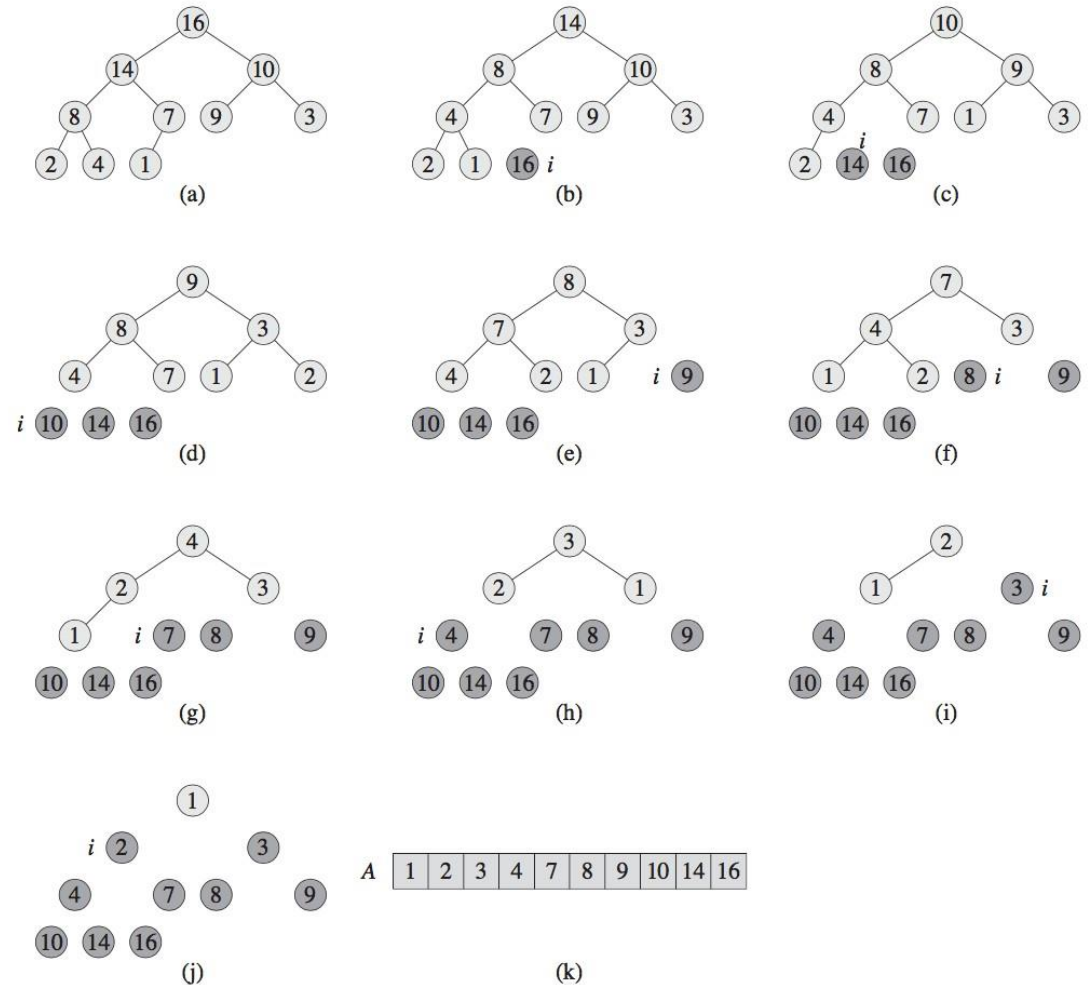
- The heapsort algorithm starts by using BUILD-MAX-HEAP to build a max-heap on the input array $A[1, \dots, n]$, where $n = A.length$.
- Since the maximum element of the array is stored at the root $A[1]$, we can put it into its correct final position by exchanging it with $A[n]$.

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Visualizing HEAPSORT

- The operation of HEAPSORT. (a) The max-heap data structure just after BUILD-MAX-HEAP has been built in line 1. (b)–(j) The max-heap just after each call of MAX-HEAPIFY in line 5, showing the value of i at that time. Only lightly shaded nodes remain in the heap. (k) The resulting sorted array A .



Running time for Heapsort

- **BUILD-MAX-HEAP** takes time $O(n)$ and each of **MAX-HEAPIFY** calls takes time $O(\log n)$.
- Therefore, the **HEAPSORT** procedure takes time $O(n \log n)$.

Priority Queue

- Heapsort is an excellent algorithm, but a good implementation of quicksort, usually beats it in practice.
- Nevertheless, the heap has its specialty:
 - most popular applications of a heap: as an efficient priority queue.
- Priority queues come in two forms:
 - max-priority queues.
 - min-priority queues.
- A *priority queue* is a data structure for maintaining a set S of elements, each with an associated value called a *key*.

Max-priority and min-priority Queue

- A *max-priority queue* supports the following operations:

INSERT(S, x) inserts the element x into the set S , which is equivalent to the operation $S = S \cup \{x\}$.

MAXIMUM(S) returns the element of S with the largest key.

EXTRACT-MAX(S) removes and returns the element of S with the largest key.

INCREASE-KEY(S, x, k) increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key value.

- Alternatively, a *min-priority queue* supports the operations INSERT, MINIMUM, EXTRACT-MIN, and DECREASE-KEY.

Max-priority queue: HEAP-MAXIMUM

- The procedure HEAP-MAXIMUM implements the MAXIMUM operation in $O(1)$ time.

HEAP-MAXIMUM(A)

1 **return** $A[1]$

Running time of HEAP-MAXIMUM

- The procedure HEAP-MAXIMUM implements the MAXIMUM operation in $O(1)$ time.

Max-priority queue: HEAP-EXTRACT-MAX

- The procedure HEAP-EXTRACT-MAX implements the EXTRACT-MAX operation.
- It is similar to the for-loop body (lines 3–5) of the HEAPSORT procedure.

```
HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2      error “heap underflow”
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size − 1
6  MAX-HEAPIFY(A, 1)
7  return max
```

Running time of HEAP-EXTRACT-MAX

- The running time of HEAP-EXTRACT-MAX is $O(\log n)$, since it performs only a constant amount of work on top of the $O(\log n)$ time for MAX-HEAPIFY.

Max-priority queue: HEAP-INCREASE-KEY

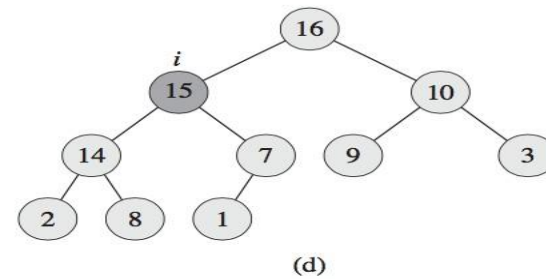
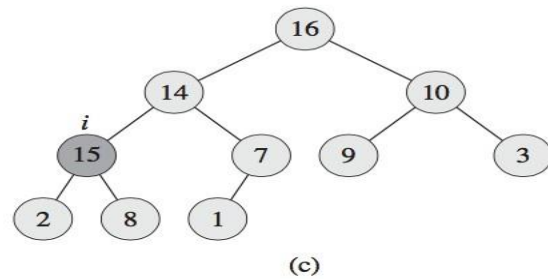
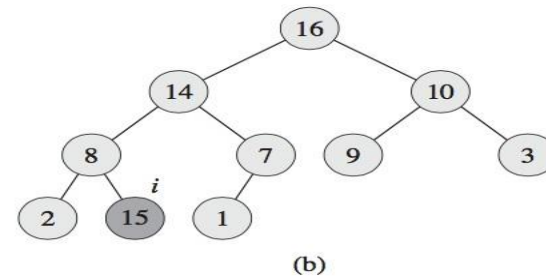
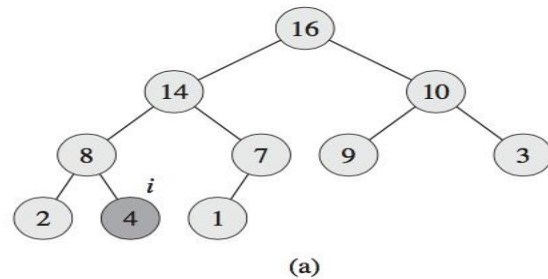
- The procedure HEAP-INCREASE-KEY implements the INCREASE-KEY operation.

HEAP-INCREASE-KEY(A, i, key)

```
1  if  $key < A[i]$ 
2      error "new key is smaller than current key"
3   $A[i] = key$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5      exchange  $A[i]$  with  $A[PARENT(i)]$ 
6       $i = PARENT(i)$ 
```

Visualizing HEAP-INCREASE-KEY

- The operation of HEAP-INCREASE-KEY. (a) The max-heap of (a) with a node whose index is i heavily shaded. (b) This node has its key increased to 15. (c) After one iteration of the while loop of lines 4–6, the node and its parent have exchanged keys, and the index i moves up to the parent. (d) The max-heap after one more iteration of the while loop. At this point, $A[\text{PARENT}(i)] \geq A[i]$. The max-heap property now holds, and the procedure terminates.



Running time for HEAP-INCREASE-KEY

- The running time of HEAP-INCREASE-KEY on an n -element heap is $O(\log n)$, since the path traced from the node updated in line 3 to the root has length $O(\log n)$.

Max-priority queue: MAX-HEAP-INSERT

- The procedure MAX-HEAP-INSERT implements the INSERT operation.

MAX-HEAP-INSERT(A, key)

1 $A.heap-size = A.heap-size + 1$

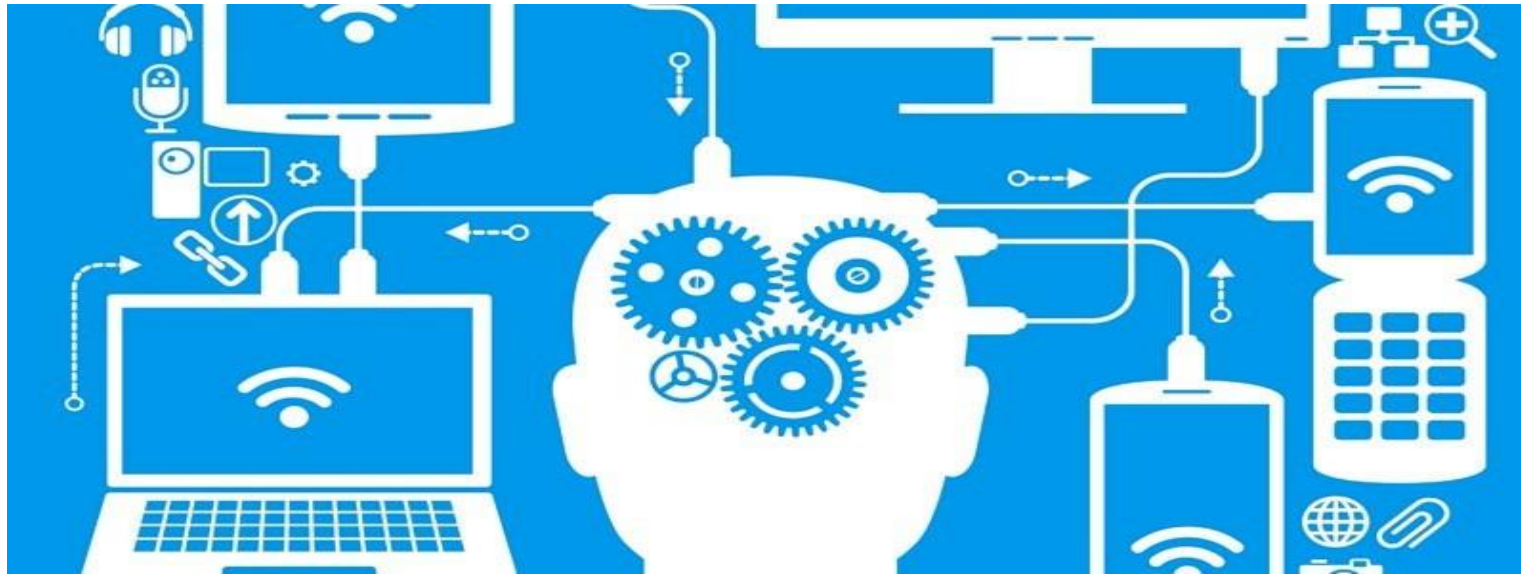
2 $A[A.heap-size] = -\infty$

3 HEAP-INCREASE-KEY($A, A.heap-size, key$)

Running time for MAX-HEAP-INSERT

- The running time of MAX-HEAP-INSERT on an n -element heap is $O(\log n)$.

In the next lecture..



Lecture 7: Probabilistic Analysis and Randomize Algorithm